

browser-adapter

Browser Adapter

The browser adapter provides an interface for browser-based UI testing. It abstracts browser automation tools behind a common contract, enabling challenge templates to interact with web applications without coupling to a specific browser driver.

BrowserAdapter Interface

Defined in `adapter_browser.go`:

```
type BrowserAdapter interface {
    Initialize(ctx context.Context, config BrowserConfig) error
    Navigate(ctx context.Context, url string) error
    Click(ctx context.Context, selector string) error
    Fill(ctx context.Context, selector, value string) error
    SelectOption(ctx context.Context, selector, value string)
        error
    IsVisible(ctx context.Context, selector string) (bool, error)
    WaitForSelector(ctx context.Context, selector string, timeout
        time.Duration) error
    GetText(ctx context.Context, selector string) (string, error)
    GetAttribute(ctx context.Context, selector, attr string)
        (string, error)
    Screenshot(ctx context.Context) ([]byte, error)
    EvaluateJS(ctx context.Context, script string) (string, error)
    NetworkIntercept(ctx context.Context, pattern string, handler
        func(req *InterceptedRequest)) error
    Close(ctx context.Context) error
    Available(ctx context.Context) bool
}
```

Method Summary

Method	Purpose
Initialize	Set up the browser with viewport size, headless mode, and extra arguments
Navigate	Load a URL in the browser tab

Method	Purpose
Click	Click the element matching a CSS selector
Fill	Type a value into an input matching a CSS selector
SelectOption	Select a dropdown option by value
IsVisible	Check if an element is visible
WaitForSelector	Block until an element appears or timeout expires
GetText	Read the text content of an element
GetAttribute	Read a named attribute from an element
Screenshot	Capture the viewport as a PNG byte slice
EvaluateJS	Execute arbitrary JavaScript and return the result
NetworkIntercept	Register a handler for matching network requests
Close	Shut down the browser and release resources
Available	Check if the automation tool is installed and reachable

Configuration Types

BrowserConfig

```
type BrowserConfig struct {
    BrowserType string `json:"browser_type" // e.g.,
                        "chromium", "firefox"
    Headless    bool    `json:"headless"`
    WindowSize [2]int  `json:"window_size" // [width, height]
    ExtraArgs   []string `json:"extra_args"`
}
```

InterceptedRequest

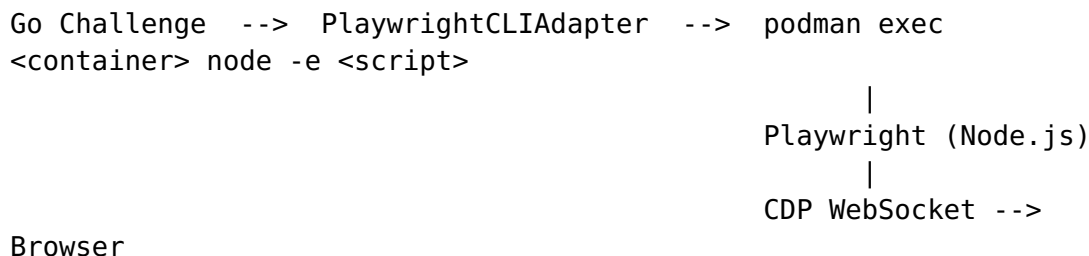
Represents a captured network request:

```
type InterceptedRequest struct {
    URL      string `json:"url"`
    Method   string `json:"method"`
    Headers  map[string]string `json:"headers"`
    Body     []byte `json:"body"`
}
```

PlaywrightCLIAdapter

The built-in implementation connects to a Chrome DevTools Protocol (CDP) endpoint and executes Playwright commands via Node.js scripts run inside a container.

Architecture



Each adapter method generates a self-contained Node.js script that:

1. Connects to the browser via `chromium.connectOverCDP(endpoint)`.
2. Accesses the existing browser context and page.
3. Performs the requested action.
4. Outputs JSON to stdout for Go to parse.

Constructor

```
adapter := userflow.NewPlaywrightCLIAdapter("ws://localhost:9222")
```

The single argument is the CDP WebSocket URL. The adapter defaults to running scripts inside a container named "playwright" via `podman exec`.

Availability Check

`Available()` converts the WebSocket URL to HTTP and performs a health check request. Returns true if the endpoint responds with a status code below 500.

Limitations

- **Network interception:** The CLI-based approach does not support persistent network interception. `NetworkIntercept()` returns nil without error, but the handler will not fire. For full network interception, a persistent-connection implementation would be needed.
- **State management:** Each method call creates a new Node.js process. Browser context and page are retrieved from the existing CDP connection on each call. This means the browser must be kept running externally (e.g., in a container).

- **Script injection safety:** Selectors and values are escaped via a helper that handles backslashes and single quotes, but complex selectors should be tested carefully.

Example: Browser Flow

```
flow := userflow.BrowserFlow{
    Name:      "login-flow",
    StartURL:  "http://localhost:3000",
    Config: userflow.BrowserConfig{
        BrowserType: "chromium",
        Headless:    true,
        WindowSize:  [2]int{1920, 1080},
    },
    Steps: []userflow.BrowserStep{
        {
            Name:      "fill-username",
            Action:     "fill",
            Selector:   "#username",
            Value:      "admin",
        },
        {
            Name:      "fill-password",
            Action:     "fill",
            Selector:   "#password",
            Value:      "secret",
        },
        {
            Name:      "click-login",
            Action:     "click",
            Selector:   "button[type=submit]",
        },
        {
            Name:      "verify-dashboard",
            Action:     "assert_visible",
            Selector:   ".dashboard",
            Screenshot: true,
        },
    },
},

challenge := userflow.NewBrowserFlowChallenge(
    "CH-BROWSER-001",
    "Login Flow",
    "Verify the login form works end-to-end",
    nil,
    adapter,
    flow,
)
```

Supported Browser Actions

The BrowserFlowChallenge dispatches the following actions:

Action	Fields Used	Behavior
navigate	Value (URL)	Loads the URL
click	Selector	Clicks the element
fill	Selector, Value	Types value into input
select	Selector, Value	Selects dropdown option
wait	Selector, Timeout	Waits for element (default 5s)
assert_visible	Selector	Fails if element is not visible
assert_text	Selector, Value	Fails if element text does not contain value
assert_url	Value	Fails if current URL does not contain value
screenshot	--	Captures viewport as PNG
evaluate_js	Value or Script	Evaluates JavaScript in browser context